TD 2 – Électronique Numérique

Logique Combinatoire

Notes et Conseils

L'objectif de la séance est multiple. Nous commençons par la construction d'opérateurs matériels élémentaires (additionneurs, multiplexeurs, comparateurs), ainsi que la manipulation algébrique d'expressions booléennes : minimisation, mise en forme canonique, etc. Les tableaux de Karnaugh constituent une autre technique tabulée de minimisation astucieuse qu'il est intéressant de connaître pour de petits circuits. Enfin, forts de la constitution des opérateurs précédents, nous pourrons aborder la conception de circuits qui en font usage, et qui dépassent la seule logique booléenne.

1 Équations logiques de circuits de base

1.1 Additionneur

1. Établir la table de vérité d'un *demi-additionneur* logique, puis les deux équations logiques associées : ce circuit additionne simplement 2 entrées booléennes a et b (donc *sans retenue entrante*). Il calcule deux sorties : la somme s et la retenue sortante c_o. Dessiner le circuit.

Solution

a	b	s	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

On trouve les équations suivantes :

$$s = a \oplus b$$
$$c_o = a \cdot b$$

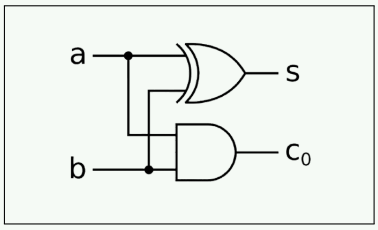


FIGURE 1 – Circuit (netlist) du demi-additionneur 1 bit

2. Établir la table de vérité d'un *additionneur complet* pour deux opérandes 1 bit. Ce dernier tient désormais compte d'une retenue entrante c_i. Établir les équations simplifiées des sorties, puis dessiner le circuit correspondant. Au préalable, on démontrera que : $\overline{x \oplus y} = x \oplus \overline{y}$.

Solution

Table de vérité :

a	b	c_i	s	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Formes brutes :

$$s = \overline{a}\overline{b}c_i + \overline{a}b\overline{c_i} + a\overline{b}\overline{c_i} + abc_i$$
$$c_o = \overline{a}bc_i + a\overline{b}c_i + ab + \overline{c_i} + abc_i \quad \text{(Note : forme brute à simplifier)}$$

Simplification :

$$s = \overline{a}(b \oplus c_i) + a(b \oplus \overline{c_i})$$
$$c_o = a \cdot b + c_i(a \oplus b)$$

Démontrons que $\overline{x \oplus y} = x \oplus \overline{y}$:

$$\begin{aligned}\overline{x \oplus y} &= \overline{x\overline{y} + \overline{x}y} \\ &= (\overline{x} + y)(x + \overline{y}) \quad \text{(De Morgan)} \\ &= \overline{x}x + \overline{x}\overline{y} + xy + y\overline{y} \\ &= 0 + \overline{x}\overline{y} + xy + 0 \\ &= x \oplus \overline{y}\end{aligned}$$

En posant $\alpha = b \oplus c_i$, on a pour la somme s :

$$s = \overline{a}\alpha + a\overline{\alpha} = a \oplus \alpha = a \oplus b \oplus c_i$$

En résumé :

$$s = a \oplus b \oplus c_i$$
$$c_o = a \cdot b + c_i(a \oplus b)$$

3. Montrer qu'un additionneur complet "1 bit" peut réutiliser le demi-additionneur de la question 1.

Solution

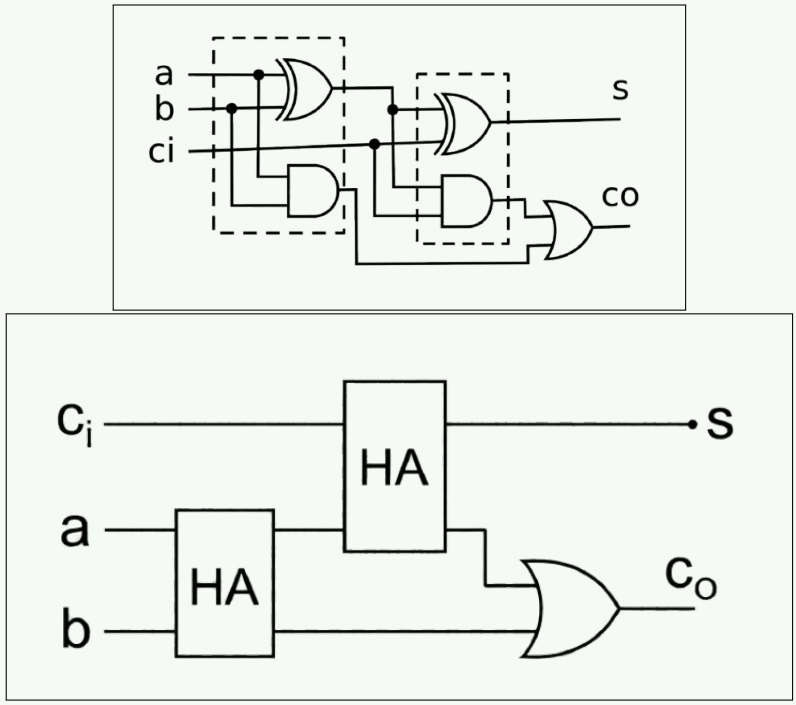


FIGURE 2 – Circuit de l'additionneur 1 bit complet (full-adder) et sa réalisation avec deux demi-additionneurs

4. Dessiner un additionneur 4 bits en réutilisant l'additionneur 1 bit.

Solution

On propose ci-dessous 2 solutions : la première ne permet pas l'utilisation de retenue entrante globale, tandis que la seconde utilise une telle retenue. La seconde solution est parfaitement régulière en terme d'aboutement, constituée de full-adders 1 bit en cascade.

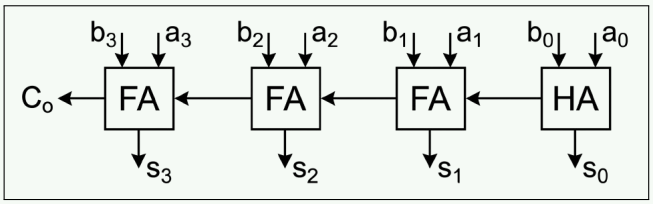


FIGURE 3 – Additionneur 4 bits sans retenue entrante globale

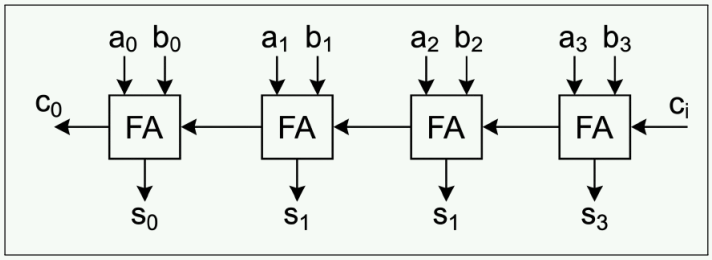


FIGURE 4 – Additionneur 4 bits avec retenue entrante (Ripple Carry Adder)

5. Quel est le chemin critique de ce dernier circuit ? On rappelle que le chemin critique est le parcours le plus long entre une entrée et une sortie combinatoire. On le mesurera en nombre de portes logiques élémentaires traversées.

Solution

Dans la réalité, chaque porte logique est pré-caractérisée par des temps de traversée donnés par le constructeur (Intel, TSMC, etc.) et il faut sommer des *temps*. L'exercice simplifie cela en comptant le nombre de portes "abstraites".
Nommons les étages de 0 à 3. Deux cas distincts :

(a) **Sans retenue entrante globale :** L'étage 0 a un chemin critique de 3 portes (2 demi-additionneurs + 1 OU). Les étages 1 à 3 ont un chemin critique de c_{in} à c_{out}, soit 2 portes chacun. Total pour les 3 étages : $3 \times 2 = 6$. Chemin critique total : $\mathbf{3 + 6 = 9}$ portes.

(b) **Avec retenue entrante globale :** Le même raisonnement conduit à un chemin critique de **7 portes** traversées (le premier étage étant un full-adder complet dès le départ).

Remarque : structurellement, 9 portes existent dans le premier cas, créant un "faux chemin critique" si l'entrée est forcée à 0.

6. Peut-on imaginer d'anticiper le calcul de la retenue ? Si oui, comment ?

1.2 Multiplexeur

La fonction d'un multiplexeur est de sélectionner, grâce à un signal de contrôle, un signal d'entrée parmi plusieurs et de transmettre sa valeur sur une sortie unique.

1. Dessiner le symbole communément admis pour représenter un multiplexeur à 2 entrées binaires (chacune codée sur 1 bit). On parle de multiplexeur "2 vers 1".
2. Calculer l'équation d'un multiplexeur "2 vers 1" pour des entrées codées sur 1 bit.
3. Lorsque les données d'entrées sont codées sur $n > 1$ bits, chacun des bits du signal multiplexé suit évidemment un chemin similaire. Dessiner un tel multiplexeur à 2 entrées, pour $n = 2$.
4. Etablir la structure du multiplexeur à m entrées, à l'aide de multiplexeurs.
5. Calculer le nombre de couches logiques traversées pour un multiplexeur m, n ("m vers 1" et dont les données sont codées sur n bits).

Solution

On parle de multiplexeur 2-vers-1 (2-to-1 MUX).

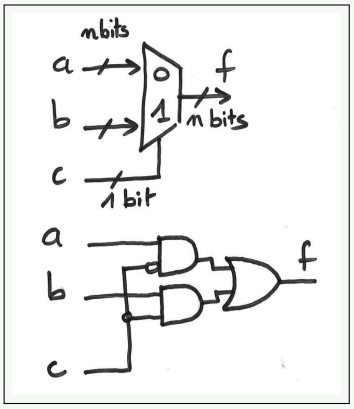


FIGURE 5 – Multiplexeur 2 vers 1, codé sur 1 bit : symbole et circuit

Pour n bits, il suffit de mettre n multiplexeurs 2-vers-1 en parallèle, tous pilotés par le même signal de sélection.

1.3 Comparateur

Soit deux données a et b quelconques, encodées par deux variables booléennes générales $(a_{n-1}, \ldots, a_0) \in \mathcal{B}^n$ et $(b_{n-1}, \ldots, b_0) \in \mathcal{B}^n$.

1. Démontrer que le résultat booléen f de comparaison d'égalité entre les deux données vaut :
$$f(a,b) = \prod_{i=0}^{i=n} \overline{a_i \oplus b_i}$$
2. Dessiner la structure du comparateur matériel pour $n = 4$.

2 Formes canoniques

Prérequis : Codage binaire et indexation des combinaisons booléennes
Pour un système à n variables logiques ordonnées, il existe 2^n combinaisons possibles de valeurs. Chaque combinaison peut être interprétée comme un nombre binaire, ce qui permet de lui associer de manière unique un indice entier compris entre 0 et $2^n - 1$. Par exemple, pour trois variables ordonnées (x, y, z), la combinaison de vérités $(1, 0, 1)$ est associée au mot binaire 101_2, qui correspond à l'indice décimal 5 : ce dernier pourra être évoqué comme le monôme m_5. Ce formalisme d'indexation numérique standardise la désignation de n'importe quel état du système et s'applique de manière générale en logique combinatoire, notamment pour l'organisation des tables de vérité, le référencement des cases d'un tableau de Karnaugh, etc.

Formes canoniques et passage de l'une à l'autre
La **première forme canonique** (ou Forme Normale Disjonctive, FND) est une somme de produits appelés mintermes (m_i), où chaque m_i est le produit logique associé à la combinaison d'indice i pour laquelle la fonction vaut 1. À l'inverse, la **deuxième forme canonique** (ou Forme Normale Conjonctive, FNC) est un produit de sommes appelées maxterms (M_i), où chaque M_i est la somme logique associée à la combinaison d'indice i pour laquelle la fonction vaut 0. Le passage d'une forme à l'autre exploite directement la complémentarité des indices au sein de l'ensemble des 2^n combinaisons. La fonction inverse $\bar{f}$ étant constituée des mintermes dont les indices sont absents de f, l'application de la double négation $(f = \overline{\bar{f}})$ et des lois de De Morgan

permet de transformer la somme de ces mintermes manquants en un produit de maxterms de mêmes indices :

$$f(x,y,z) = m_1 + m_3 + m_5 \quad \text{(la fonction vaut 1 pour les indices 1, 3 et 5)}$$
$$\bar{f}(x,y,z) = m_0 + m_2 + m_4 + m_6 + m_7 \quad \text{(indices restants parmi les } 2^3 \text{ possibles)}$$
$$f(x,y,z) = \overline{\bar{f}(x,y,z)} = M_0 \cdot M_2 \cdot M_4 \cdot M_6 \cdot M_7 \quad \text{(par passage au dual via De Morgan)}$$

Ainsi, commuter entre les deux formes canoniques revient simplement à inverser la nature de l'opérateur principal (somme ou produit) en basculant sur l'ensemble complémentaire des indices de l'univers des combinaisons. *Note : Ces formes ne sont pas simplifiées.*

Établir les deux formes canoniques des fonctions suivantes :

1. $F_1 = xy + yz + xz$
2. $F_2 = x + yz + \overline{yz}t$

Solution

1. $F_1 = XY + YZ + XZ$

X	Y	Z	F_1
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- Première forme canonique

$$F_1 = \overline{X}YZ + X\overline{Y}Z + XY\overline{Z} + XYZ$$

- Seconde forme canonique

$$F_1 = (X + Y + Z)(X + Y + \overline{Z})(X + \overline{Y} + Z)(\overline{X} + Y + Z)$$

2. $F_2 = X + YZ + \overline{Y}\,\overline{Z}\,T$

X	Y	Z	T	F_2
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

FIGURE 6 – Solution pour F_1

- Première forme canonique

$$F_2 = \overline{X}\,\overline{Y}\,\overline{Z}T + \overline{X}\,Y\,Z\overline{T} + \overline{X}\,Y\,ZT + X\,\overline{Y}\,\overline{Z}\,\overline{T} + X\,\overline{Y}\,\overline{Z}\,T + X\,\overline{Y}\,Z\overline{T} + X\overline{Y}\,ZT + XY\overline{Z}\,\overline{T} + XY\overline{Z}\,T + XYZ\overline{T} + XYZT$$

- Seconde forme canonique

$$F_2 = (X + Y + Z + T)(X + Y + \overline{Z} + T)(X + Y + \overline{Z} + \overline{T})(X + \overline{Y} + Z + T)(X + \overline{Y} + Z + \overline{T})$$

FIGURE 7 – Solution pour F_2

3 Quelques formulations booléennes

Écrire sous la première forme canonique (somme de mintermes) les fonctions définies par les propositions suivantes :

1. $F(a,b,c) = 1$ si et seulement si aucune des variables a, b, c ne prend la valeur 1.
2. $F(a,b,c) = 1$ si et seulement si au plus une des variables a, b, c prend la valeur 0.

Solution

1. $f(A,B,C) = \overline{A}\,\overline{B}\,\overline{C}$
2. $f(A,B,C) = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$

FIGURE 8 – Solutions des formulations booléennes

4 Simplifications algébriques

Simplifier algébriquement les fonctions suivantes :

1. $F = \overline{x}\overline{y} + xy + \overline{x}y$
2. $F = (x + \overline{y})(x\overline{y} + z)z$

Solution

1. $F = \overline{x}(\overline{y} + y) + xy = \overline{x}(1) + xy = \overline{x} + xy = (\overline{x} + x)(\overline{x} + y) = \mathbf{\overline{x} + y}$
2. $F = (x + \overline{y})(x\overline{y}z + z) = (x + \overline{y})z(x\overline{y} + 1) = (x + \overline{y})z(1) = \mathbf{(x + \overline{y})z}$

5 Simplifications par tableaux de Karnaugh

Simplifier, par la méthode des tableaux de Karnaugh, les fonctions booléennes suivantes. Dessiner le circuit correspondant et déterminer son chemin critique, en supposant que toutes les portes présentent un temps de propagation de 5 ns.

5.1 K-map à 3 variables

1. $F(a,b,c) = \overline{a}\overline{b}c + \overline{a}b\overline{c} + ab\overline{c}$
2. $F(a,b,c) = \overline{a}b\overline{c} + \overline{a}bc + ab\overline{c}$
3. $F(a,b,c) = \overline{a}\overline{b}c + a\overline{b}\overline{c} + ab\overline{c}$, sachant que la valeur de F pour les combinaisons $\overline{a}bc$ et abc est indifférente (donc care X).

Solution

1. $F(a,b,c) = \overline{a}\overline{b}c + b\overline{c}$
2. $F(a,b,c) = \overline{a}b + b\overline{c} = b(\overline{a} + \overline{c})$
3. $F(a,b,c) = \overline{a}c + a\overline{c} = a \oplus c$ (en utilisant les cares pour former des groupes de 2 optimaux).

5.2 K-map à 4 variables

1. $F(a,b,c,d) = \bar{a}b\bar{c}\bar{d} + \bar{a}b\bar{c}d + \bar{a}bcd + \bar{a}bc\bar{d}$
2. $F(a,b,c,d) = \bar{a}\bar{b}\bar{c}d + \bar{a}\bar{b}cd + a\bar{b}\bar{c}d + a\bar{b}cd$

Solution

1. En factorisant ou via le tableau, on isole $a = 0$ et $b = 1$: $\mathbf{F = \bar{a}b}$
2. On isole $b = 0$ et $d = 1$: $\mathbf{F = \bar{b}d}$

5.3 K-map à 5 variables

$F(a,b,c,d,e) = \bar{a}\bar{b}\bar{c}\bar{d}\bar{e} + \bar{a}\bar{b}c\bar{d}\bar{e} + \bar{a}b\bar{c}\bar{d}\bar{e} + \bar{a}bc\bar{d}\bar{e} + ab\bar{c}\bar{d}\bar{e} + ab\bar{c}d\bar{e} + abcd\bar{e} + abc\bar{d}\bar{e} + a\bar{b}\bar{c}\bar{d}\bar{e} + a\bar{b}c\bar{d}\bar{e} + \bar{a}b\bar{c}\bar{d}e + \bar{a}bc\bar{d}e + ab\bar{c}\bar{d}e + ab\bar{c}de + abcde + abc\bar{d}e$

Solution

Le tableau à 5 variables se visualise comme deux tableaux de 4 variables ($a = 0$ et $a = 1$) superposés. On identifie 3 groupes de 8 cases adjacentes (en tenant compte de la symétrie entre les deux plans) :

$$\mathbf{F(a,b,c,d,e) = ab + b\bar{d} + \bar{d}\bar{e}}$$

6 Différentes utilisations du OU exclusif

1. Rappeler la table de vérité du OU exclusif, ainsi que son symbole (américain).
2. Dessiner la fonction OU-exclusif à partir de portes logiques ET, OU et NON.
3. Le OU-exclusif possède de nombreuses propriétés intéressantes. Compléter les énoncés suivants :
   (a) Sa sortie vaut 1 si exactement une des entrées vaut 1 : $x \oplus y =$??+??
   (b) On peut le considérer comme un comparateur d'égalité pour 1 bit : $x \oplus y =$ SI $(x == y)$ ALORS ? SINON ?
   (c) On peut le considérer comme un inverseur conditionnel : $x \oplus y =$ SI (?) ALORS ? SINON ?
   (d) On peut enfin le considérer comme un additionneur modulo 2.

Solution

1. $x \oplus y = x\overline{y} + \overline{x}y$
2. $x \oplus y =$ SI $(x == y)$ ALORS 0 SINON 1 (C'est l'inverse de l'égalité, c'est un détecteur de différence).
3. $x \oplus y =$ SI $(y == 1)$ ALORS $\overline{x}$ SINON x. (Si $y = 0$, $x \oplus 0 = x$; si $y = 1$, $x \oplus 1 = \overline{x}$).

7 Le jeu des cambrioleurs

Le principe du jeu [1] est le suivant : un jeton est introduit par le joueur en haut du dispositif. Le but est de faire circuler ce jeton dans le dispositif, jusqu'à ce qu'il apparaisse en sortie. Dans l'intervalle, le jeton peut rester coincé à différents endroits (ou "niveaux") : seul un code secret correctement introduit à cet endroit permet au jeton de continuer son parcours. Sinon, l'utilisateur voit le code d'erreur `0xDEAD` apparaître en sortie. Chaque code secret, associé au niveau i, est codé sur 4 bits.

1. Imaginer un circuit combinatoire qui simule ce jeu. On utilisera pour cela différents circuits connus : multiplexeurs et comparateurs. Enrichir votre circuit avec d'autres circuits combinatoires afin d'indiquer le nombre de niveaux franchis correctement.
2. Sachant que l'on peut envoyer 1 million de codes par seconde, combien de temps faudrait-il, au pire cas, pour tester toutes les combinaisons possibles, dans le cas où le nombre de niveaux est 9 ? Refaites le calcul pour des codes sur 5 bits.

Cet exercice ne nécessite pas de formules complexes, mais une réflexion architecturale.

1. Le jeu est inspiré du jeu "Dix de Chute" des années 70.

Solution

1. Architecture :
Le but est de manipuler des circuits pour réaliser un “chemin de données”. On choisit de représenter le jeton par un vecteur de bits de taille $n = 16$ (à cause du code `0xDEAD`).
À chaque niveau, un comparateur vérifie si le code d'entrée correspond au code secret. La sortie du comparateur pilote un multiplexeur : si le code est bon, le jeton actuel passe au niveau suivant ; sinon, la valeur `0xDEAD` est injectée et propagée jusqu'à la sortie. Un compteur peut être incrémenté à chaque comparaison réussie.

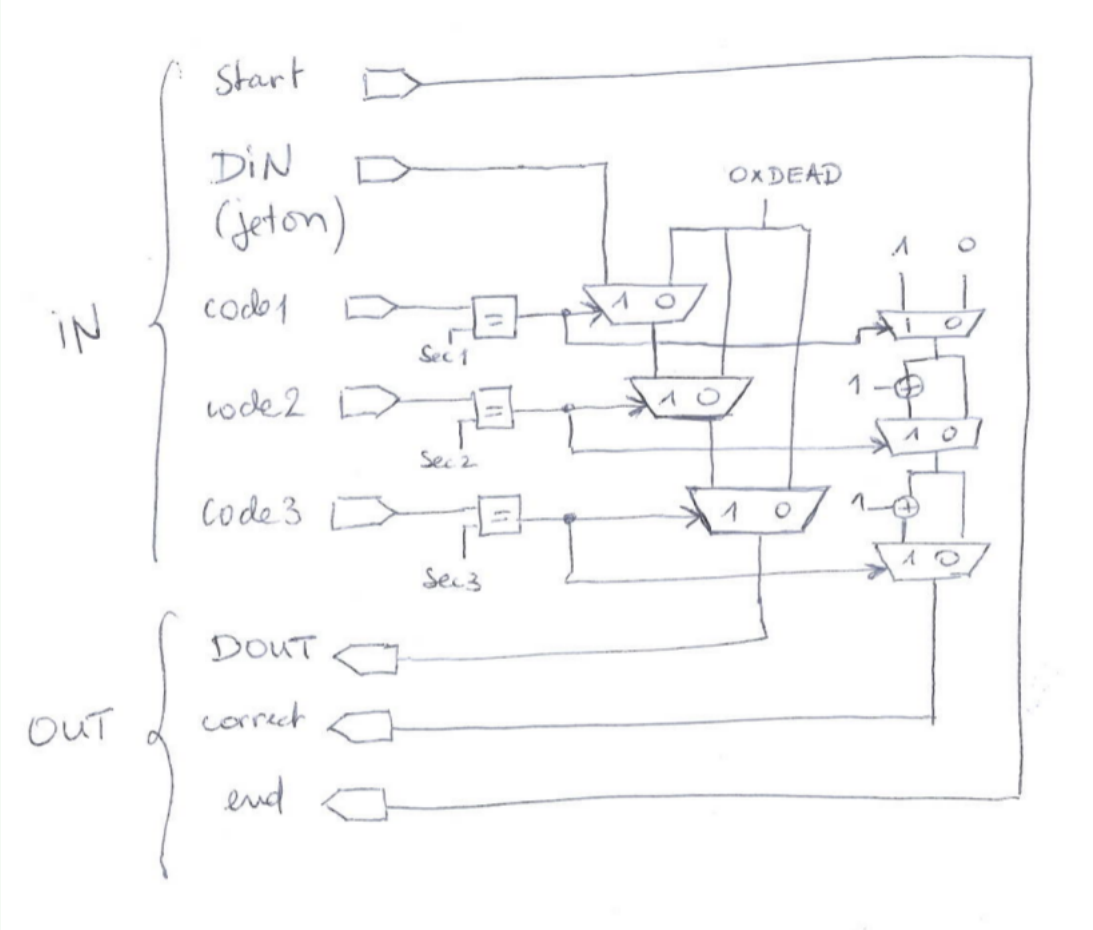


FIGURE 9 – Schéma de principe du jeu des cambrioleurs

2. Calcul de complexité :
Pour 9 niveaux avec des codes de 4 bits : il y a $(2^4)^9 = 2^{36}$ combinaisons à tester.
À 10^6 tests/seconde, le temps au pire cas est :

$$\frac{2^{36}}{10^6} \approx 68\,719 \text{ secondes} \approx \textbf{19} \text{ heures}$$

Pour des codes sur 5 bits : $(2^5)^9 = 2^{45}$ combinaisons.

$$\frac{2^{45}}{10^6} \approx 35\,184\,372 \text{ secondes} \approx \textbf{407} \text{ jours}$$

8 Vu-mètre

On se propose de réaliser un “contrôleur de vu-mètre” : c’est un dispositif qui permet de visualiser une grandeur numérique (un volume sonore par exemple) au moyen d’une barrette lumineuse qui s’étire plus ou moins en fonction de la grandeur.

1. Schématiser le principe d’ensemble dans le cas où la grandeur à visualiser est codée sur 3 bits.
2. À partir du tableau de Karnaugh, trouver une réalisation possible.
3. On suppose que l’on dispose désormais d’un composant complexe de décodage (un décodeur 3 vers 8). Rappeler la table de vérité de ce circuit. Utiliser alors ce décodeur pour proposer une nouvelle réalisation.

Solution

Attention : lorsqu'on envoie le code `000`, on ne veut *aucune* LED allumée. Quand on envoie le code `111` (7), on veut voir 7 LEDs allumées, numérotées de L_0 à L_6 (et non 8).
Table de vérité du Vu-mètre :

e_2	e_1	e_0	L_0	L_1	L_2	L_3	L_4	L_5	L_6
0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0
0	1	0	1	1	0	0	0	0	0
0	1	1	1	1	1	0	0	0	0
1	0	0	1	1	1	1	0	0	0
1	0	1	1	1	1	1	1	0	0
1	1	0	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1

Simplification (K-map) :

$$
\begin{aligned}
L_0 &= e_2 + e_1 + e_0 \\
L_1 &= e_2 + e_1 \\
L_2 &= \overline{e_2} e_1 + e_2 \overline{e_1} e_0 \quad \text{(ou forme équivalente)} \\
L_3 &= e_2 \\
L_4 &= e_2 e_1 + e_2 \overline{e_1} e_0 \\
L_5 &= e_2 e_1 \\
L_6 &= e_2 e_1 e_0
\end{aligned}
$$

Réalisation avec un décodeur 3 vers 8 :
La sortie C_i d'un décodeur s'allume (vaut 1) uniquement lorsque la valeur binaire d'entrée correspond à i.

e_2	e_1	e_0	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Solution

On remarque que L_k doit être allumée pour toutes les valeurs d'entrée strictement supérieures à k. Ainsi :

$$\begin{aligned}
L_0 &= C_1 + C_2 + C_3 + C_4 + C_5 + C_6 + C_7 \\
L_1 &= C_2 + C_3 + C_4 + C_5 + C_6 + C_7 \\
L_2 &= C_3 + C_4 + C_5 + C_6 + C_7 \\
&\vdots \\
L_5 &= C_6 + C_7 \\
L_6 &= C_7
\end{aligned}$$

On peut réaliser cela en cascade avec des portes OU, ce qui est très élégant et modulaire. Seul le cas du zéro (L_0 éteinte quand $C_0 = 1$) est géré naturellement car C_0 n'est pas inclus dans la somme de L_0.

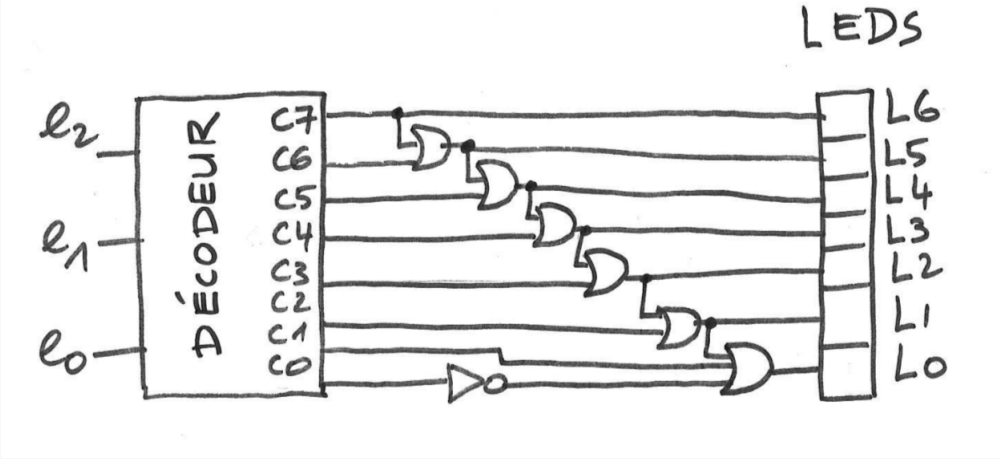


FIGURE 10 – Réalisation du Vu-mètre à l'aide d'un décodeur et de portes OU